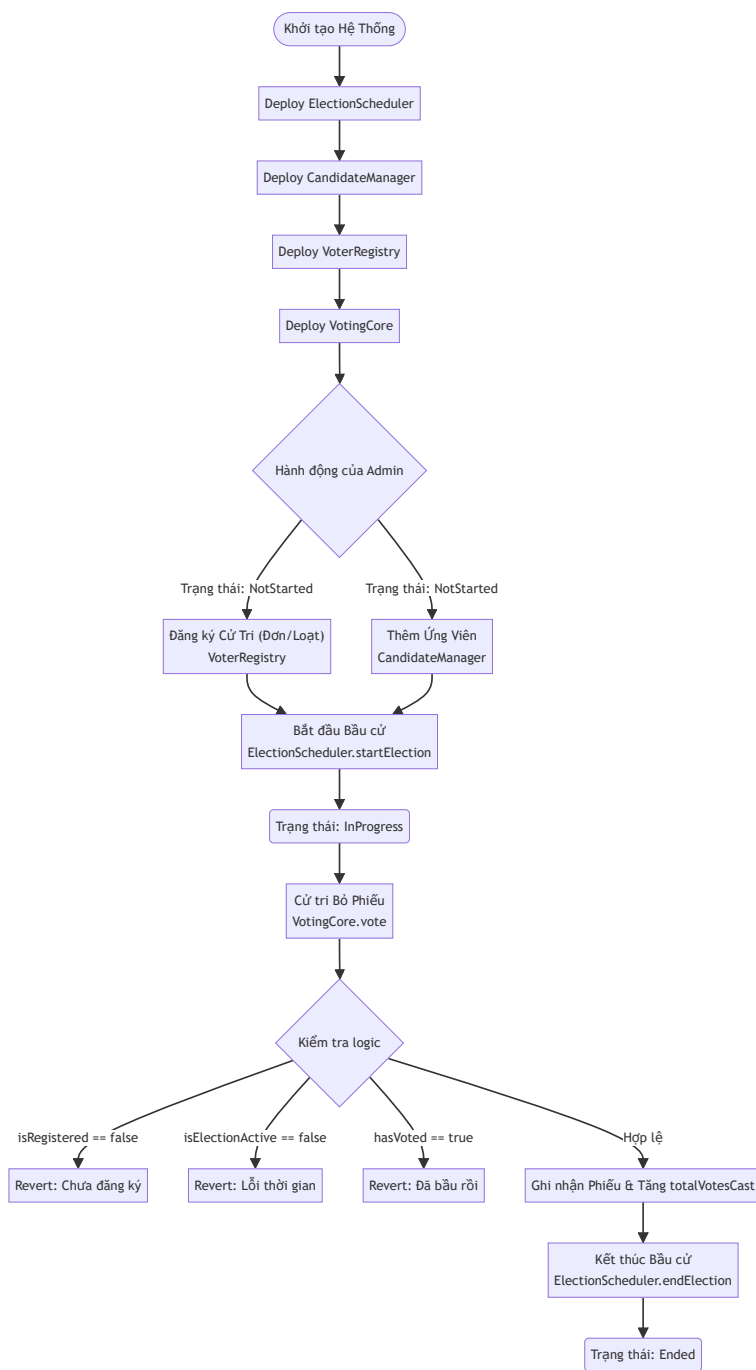


TÀI LIỆU KỸ THUẬT: VÒNG ĐÒI BẦU CỬ SMART CONTRACT

Tài liệu này cung cấp cái nhìn chuyên sâu và phân tích logic luồng chạy thực tế của hệ thống Smart Contract Bầu Cử Blockchain, dựa trên mã nguồn bản chính thức (Final Release) đã được kiểm thử và tích hợp trơn tru (VotingCore.sol, ElectionScheduler.sol, VoterRegistry.sol, CandidateManager.sol).

1. Sơ Đồ Tổng Quan Hệ Thống



2. Phân Tích Chi Tiết Từng Giai Đoạn

Giai Đoạn 1: Tiền Bầu Cử (Pre-Election / Setup)

Đây là giai đoạn khởi tạo, khi trạng thái hợp đồng `ElectionScheduler` đang là `NotStarted`.

- **Đăng ký cử tri:** Module `VoterRegistry` cung cấp 2 phương thức cực kỳ linh hoạt:
 - `registerVoter(address)` : Đăng ký từng người.
 - `batchRegisterVoters(address[])` : Đăng ký hàng loạt để tối ưu phí gas. Vòng lặp tối ưu hoá với `unchecked`.
 - *Điều kiện kiểm tra:* `onlyAdmin` (Chỉ admin thực hiện), `onlyBeforeElection` (Chỉ thực hiện khi `NotStarted`).
 - *Kiểm tra tính hợp lệ:* `require(_voter != address(0), "Dia chi cu tri khong hop le")` và `require(!registeredVoters[_voter], "Cu tri da duoc dang ky")`.

Giai Đoạn 2: Mở Hòm Phiếu (Election In Progress)

Admin gọi hàm `startElection()`. Hệ thống ghi nhận `startTime = block.timestamp` và chuyển state sang `InProgress`.

- **Cử tri Bỏ phiếu:** Hàm `vote(uint _candidateId)` trong `VotingCore.sol`.
 - *Kiểm tra danh sách trắng (Registry):*
`require(voterRegistry.isRegistered(msg.sender), "Cu tri chua duoc dang ky")`. Chỉ những ví đã được duyệt ở Giai đoạn 1 mới được bỏ phiếu.
 - *Kiểm tra trạng thái (Scheduler):*
`require(electionScheduler.isElectionActive(), "Cuoc bau cu chua bat dau hoac da ket thuc")`.
 - *Kiểm tra bỏ phiếu kép (Core):* `require(!hasVoted[msg.sender], "Ban da tham gia bau cu roi")`. Chống gian lận Double-voting tuyệt đối.
 - *Hành động:* Cập nhật `hasVoted[msg.sender] = true`, gọi `candidateManager.incrementVote()` và tăng biến `totalVotesCast++`.

Giai Đoạn 3: Kết Thúc & Chốt Số (Post-Election)

Admin gọi `endElection()`. Trạng thái chuyển vĩnh viễn sang `Ended`, ghi nhận `endTime = block.timestamp`. Từ thời điểm này, mọi lệnh gọi `vote()` đều bị Revert.

3. Bảng Tổng Hợp Các Sự Kiện (Events Log)

Mọi hành động quan trọng làm thay đổi State của Blockchain đều được minh bạch hoá qua các sự kiện (Events) sau:

Contract	Tên Event	Tham số phát ra (Payload)	Ý nghĩa Bảo mật
ElectionScheduler	ElectionStarted	uint256 startTime	Lưu vết thời gian chính xác (tính đến giây) lúc bầu cử bắt đầu.
ElectionScheduler	ElectionEnded	uint256 endTime	Đánh dấu mốc thời gian hòm phiếu bị niêm phong.
ElectionScheduler	StateChanged	ElectionState newState	Công khai sự thay đổi trạng thái của hệ thống.
VotingCore	Voted	address indexed voter, uint256 candidateId	Cung cấp bằng chứng không thể chối cãi về việc một địa chỉ ví đã bầu cho ai. Dữ liệu được index để dễ dàng truy vấn.
VoterRegistry	VoterRegistered	address indexed voter	Theo dõi danh sách những người được cấp quyền bỏ phiếu.

4. Phân Tích Tính Minh Bạch Và An Toàn (Security Review)

Kiến trúc hiện tại của dự án đáp ứng xuất sắc các tiêu chuẩn bảo mật cơ bản của một Dapp:

1. **Phân quyền truy cập (Access Control):** Modifier `onlyAdmin` được thiết kế chuẩn xác để bảo vệ các hàm nhạy cảm (`startElection` , `endElection` , `registerVoter`).
2. **Xác thực đa tầng (Multi-layer Validation):** Trong hàm `vote()` , hệ thống kiểm tra qua 3 lớp phòng ngự: Lớp thời gian (Scheduler), Lớp danh tính (Registry), và Lớp trạng thái cá nhân (hasVoted) trước khi thực sự tính phiếu. Thiết kế siêu việt!
3. **Tối ưu vòng lặp (Gas Optimization):** Hàm `batchRegisterVoters` sử dụng khối `unchecked { ++index; }` để giảm tối đa chi phí Gas khi duyệt mảng lớn.